

style.css (inside assets folder)

```
/* === Core Layout === */
```

```
body {  
    margin: 0;  
    font-family: 'Segoe UI', sans-serif;  
    background: linear-gradient(to bottom right, #0f0f0f, #1a1a1a);  
    color: #e0e0e0;  
}
```

```
/* === Symbolic Card Container === */
```

```
.card {  
    border-radius: 12px;  
    padding: 1.2rem;  
    margin: 1rem;  
    background: rgba(255, 255, 255, 0.05);  
    -webkit-backdrop-filter: blur(8px);  
    backdrop-filter: blur(8px);  
    box-shadow: 0 0 12px rgba(0, 0, 0, 0.4);  
    transition: transform 0.3s ease, box-shadow 0.3s ease;  
    cursor: pointer;  
}
```

```
.card:hover {  
    transform: scale(1.03);  
    box-shadow: 0 0 18px rgba(255, 255, 255, 0.2);  
}
```

```
/* === Elemental Themes for Cards === */  
  
.card[data-element="Air"] { border-left: 4px solid #a8e6cf; }  
.card[data-element="Fire"] { border-left: 4px solid #ff8b94; }  
.card[data-element="Water"] { border-left: 4px solid #69b7ff; }  
.card[data-element="Earth"] { border-left: 4px solid #c3b091; }  
.card[data-element="Aether"] { border-left: 4px solid #d1c4e9; }  
.card[data-element="Metal"] { border-left: 4px solid #b0bec5; }  
.card[data-element="Wood"] { border-left: 4px solid #aed581; }  
.card[data-element="Light"] { border-left: 4px solid #fff176; }  
.card[data-element="Shadow"] { border-left: 4px solid #616161; }
```

```
/* === Lore Text Styling === */  
  
.lore {  
  font-style: italic;  
  font-size: 0.95rem;  
  color: #cccccc;  
  margin-top: 0.5rem;  
}
```

```
/* === Communion Seeker Overlay === */  
  
#communion-seeker {  
  position: fixed;  
  bottom: 2rem;  
  right: 2rem;  
  background: radial-gradient(circle, #222, #000);
```

```
border: 2px solid #888;
border-radius: 50%;
width: 80px;
height: 80px;
display: flex;
align-items: center;
justify-content: center;
color: #fff;
font-size: 0.75rem;
text-align: center;
animation: pulse 3s infinite;
}
```

```
/* === Pulse Animation === */
```

```
@keyframes pulse {
  0% { box-shadow: 0 0 0px rgba(255,255,255,0.2); }
  50% { box-shadow: 0 0 12px rgba(255,255,255,0.4); }
  100% { box-shadow: 0 0 0px rgba(255,255,255,0.2); }
}
```

```
/* === Elemental Triangle Styling === */
```

```
.triangle-group polygon[data-element="Air"] {
  stroke: #a8e6cf;
  fill: rgba(168, 230, 207, 0.2);
}
```

```
.triangle-group polygon[data-element="Fire"] {  
  stroke: red;  
  fill: rgba(255, 0, 0, 0.2);  
}
```

```
.triangle-group polygon[data-element="Water"] {  
  stroke: blue;  
  fill: rgba(0, 0, 255, 0.2);  
}
```

```
.triangle-group polygon[data-element="Earth"] {  
  stroke: #c3b091;  
  fill: rgba(195, 176, 145, 0.2);  
}
```

```
.triangle-group polygon[data-element="Aether"] {  
  stroke: #d1c4e9;  
  fill: rgba(209, 196, 233, 0.2);  
}
```

```
.triangle-group polygon[data-element="Metal"] {  
  stroke: #b0bec5;  
  fill: rgba(176, 190, 197, 0.2);  
}
```

```
.triangle-group polygon[data-element="Wood"] {
```

```
stroke: #aed581;
fill: rgba(174, 213, 129, 0.2);
}
```

```
.triangle-group polygon[data-element="Light"] {
stroke: #fff176;
fill: rgba(255, 241, 118, 0.2);
}
```

```
.triangle-group polygon[data-element="Shadow"] {
stroke: #616161;
fill: rgba(97, 97, 97, 0.2);
}
```

```
/* === Triangle Hover Glow === */
.triangle-group polygon:hover {
stroke-width: 2;
filter: drop-shadow(0 0 6px rgba(255, 255, 255, 0.3));
cursor: pointer;
}
```

```
/* === Symbol Text Styling === */
.triangle-group text {
font-family: 'Segoe UI', sans-serif;
fill: #e0e0e0;
font-size: 14px;
```

```

    pointer-events: none;
}

.nested-triangle {
    stroke: gold;
    stroke-width: 1;
    fill: rgba(255, 215, 0, 0.1);
}

.triangle-group polygon:hover {
    stroke-width: 2;
    filter: drop-shadow(0 0 6px rgba(255, 255, 255, 0.3));
    cursor: pointer;
}

.triangle-group text {
    font-family: 'Segoe UI', sans-serif;
    fill: #e0e0e0;
    font-size: 14px;
    pointer-events: none;
}

```

triangle_cards.js (inside assets folder)

```

    window.addEventListener("load", () => {
    setTimeout(() => {
        const loreContainer = document.getElementById("symbolic-card");
        const triangles = document.querySelectorAll(".triangle");

```

```

if (!loreContainer || triangles.length === 0) {

  console.warn(" ⚠ Lore container or triangles not found.");

  return;

}

triangles.forEach(tri => {

  tri.addEventListener("click", () => {

    const name = tri.getAttribute("data-name") || "Unknown";

    const element = tri.getAttribute("data-element") || "Unknown";

    const symbol = tri.getAttribute("data-symbol") || "";

    const lore = tri.getAttribute("data-lore") || "Lore not available.";

    loreContainer.innerHTML = `

      <h3>${symbol} (${element})</h3>

      <h4>${name}</h4>

      <p>${lore}</p>

    `;

  });

});

}, 500);

});

```

lore.json (inside data folder)

```

{
  "triangles": [
    {

```

```
"id": 1,

"name": "Triangle of Air",

"element": "Air",

"mantra": "Vayu Namah",

"symbol": "🌀 ",

"lore": "Breath is the bridge between thought and form. Air carries the whisper of intention.",

"rotation": 0

},

{

  "id": 2,

  "name": "Triangle of Fire",

  "element": "Fire",

  "mantra": "Agni Namah",

  "symbol": "🔥 ",

  "lore": "Fire transforms. It consumes illusion and reveals the radiant core of being.",

  "rotation": 40

},

{

  "id": 3,

  "name": "Triangle of Water",

  "element": "Water",

  "mantra": "Jala Namah",

  "symbol": "💧 ",

  "lore": "Water remembers. It flows through memory, emotion, and the sacred pulse of life.",
```

```
    "rotation": 80
  },
  {
    "id": 4,
    "name": "Triangle of Earth",
    "element": "Earth",
    "mantra": "Prithvi Namah",
    "symbol": "🌍",
    "lore": "Earth holds. It anchors the soul in form, offering stability and sacred geometry.",
    "rotation": 120
  },
  {
    "id": 5,
    "name": "Triangle of Aether",
    "element": "Aether",
    "mantra": "Akasha Namah",
    "symbol": "💠",
    "lore": "Aether is the unseen field. It binds all elements in communion and silence.",
    "rotation": 160
  },
  {
    "id": 6,
    "name": "Triangle of Metal",
    "element": "Metal",
    "mantra": "Loha Namah",
```

```
"symbol": " ⚙️ ",
"lore": "Metal reflects. It is the mirror of karma, forged by discipline and clarity.",
"rotation": 200
},
{
  "id": 7,
  "name": "Triangle of Wood",
  "element": "Wood",
  "mantra": "Vriksha Namah",
  "symbol": " 🌳 ",
  "lore": "Wood grows. It spirals upward, connecting root to sky in sacred expansion.",
  "rotation": 240
},
{
  "id": 8,
  "name": "Triangle of Light",
  "element": "Light",
  "mantra": "Jyoti Namah",
  "symbol": " ☀️ ",
  "lore": "Light reveals. It is the language of truth, illuminating the path of the seeker.",
  "rotation": 280
},
{
  "id": 9,
  "name": "Triangle of Shadow",
  "element": "Shadow",
```

```

    "mantra": "Chhaya Namah",

    "symbol": "🕒",

    "lore": "Shadow conceals. It is the womb of transformation, where the unseen
prepares to emerge.",

    "rotation": 320

}

]

}

```

app.py (inside root of project)

```

import streamlit as st

from triangle_utils import isosceles_triangle
from metadata import load_triangle_meta
from streamlit_option_menu import option_menu


# 📐 Sacred Geometry SVG Generator

def svg_triangle(points, symbol, element, name, lore):

    point_str = " ".join([f"{x},{y}" for x, y in points])

    cx, cy = points[2] # tip of triangle


    return f"""

<g class="triangle-group">

  <polygon points="{point_str}" class="triangle"

    data-symbol="{symbol}"

    data-element="{element}"

    data-name="{name}"

```

```

    data-lore="{lore.replace('"' , '&quot;')}" />

<circle cx="{cx}" cy="{cy}" r="10" fill="none" stroke="gold" stroke-width="2" />

<text x="{cx}" y="{cy + 4}" text-anchor="middle">{symbol}</text>

</g>

""""

```

🛠 Page config

```
st.set_page_config(layout="wide")
```

📦 Load triangle metadata

```
TRIANGLE_META = load_triangle_meta()
```

🎨 Inject CSS

```
def local_css(file_name):
```

```
    try:
```

```
        with open(file_name, encoding="utf-8") as f:
```

```
            st.markdown(f"<style>{f.read()}</style>", unsafe_allow_html=True)
```

```
    except FileNotFoundError:
```

```
        st.error(f"Missing CSS file: {file_name}")
```

```
local_css("assets/style.css")
```

📖 Sidebar navigation

```
selected_element = option_menu(
```

```
    menu_title="Elemental Realms",
```

```

    options=["All", "Air", "Fire", "Water", "Earth", "Aether", "Metal", "Wood", "Light",
"Shadow"],

    icons=["globe", "wind", "fire", "droplet", "tree", "stars", "gear", "leaf", "sun", "moon"],

    orientation="vertical",

    key="element_selector"

)

```

🔍 Filter triangles by element

```

filtered_meta = (
    TRIANGLE_META if selected_element == "All"
    else [m for m in TRIANGLE_META if m["element"] == selected_element]
)

```

if not filtered_meta:

```

    st.warning(f"No triangles found for {selected_element} realm.")

```

🌟 View selector

```

view = st.radio("Choose view", ["Disintegrated", "Integrated", "Symbolic"],
key="view_selector")

```

▲ Triangle rendering

```

svg_elements = []

```

for meta in filtered_meta:

```

    center = (300, 300) if view == "Symbolic" else (200 + meta["id"] * 20, 200 + meta["id"]
* 10)

```

```

    points = isosceles_triangle(center=center, rotation=meta["rotation"])

```

```

    triangle_svg = svg_triangle(

```

```

        points,
        symbol=meta["symbol"],
        element=meta["element"],
        name=meta["name"],
        lore=meta["lore"]
    )
    svg_elements.append(triangle_svg)

```

🖼️ Render SVG

```

st.markdown(f"""
<svg width="600" height="600">
    {' '.join(svg_elements)}
</svg>
""", unsafe_allow_html=True)

```

📜 Symbolic lore display

```

if view == "Symbolic":

```

```

    st.markdown("### 🧬 Symbolic Lore of the Nine Triangles")

```

```

    triangle_names = [meta["name"] for meta in filtered_meta]

```

```

    selected_triangle = st.selectbox("Choose a triangle to reveal its lore", triangle_names,
key="triangle_selector")

```

```

    selected_meta = next((m for m in filtered_meta if m["name"] == selected_triangle),
None)

```

```

    if selected_meta:

```

```

st.markdown(f"""
<div class="symbolic-card">

    <h3>{selected_meta['symbol']} ({selected_meta['element']})</h3>

    <h4>{selected_meta['name']}</h4>

    <p>{selected_meta['lore']}</p>

</div>

""", unsafe_allow_html=True)

```

metadata.py (inside root of Project)

```

import json
import os

def load_triangle_meta():
    path = os.path.join("data", "lore.json")
    try:
        with open(path, "r", encoding="utf-8") as f:
            content = f.read()
            if not content.strip():
                raise ValueError("Empty JSON file.")
            raw = json.loads(content)

            meta = raw.get("triangles", [])
            if not isinstance(meta, list):
                raise ValueError("Expected 'triangles' to be a list.")

```

```

# Inject rotation if missing

for i, item in enumerate(meta):

    item.setdefault("rotation", i * 40)


return meta


except (FileNotFoundError, ValueError, json.JSONDecodeError) as e:

    print(f" ⚠️ Failed to load lore.json: {e}. Falling back to default metadata.")

    return default_triangle_meta()


def default_triangle_meta():

    symbols = [

        ("Wisdom", "Air", "Clarity of thought and the breath of insight."),
        ("Courage", "Fire", "The spark that ignites transformation."),
        ("Harmony", "Water", "Flowing unity and emotional resonance."),
        ("Structure", "Earth", "Foundation, form, and sacred geometry."),
        ("Vision", "Aether", "The unseen pattern behind all things."),
        ("Balance", "Metal", "Precision, reflection, and inner alignment."),
        ("Flow", "Wood", "Growth, adaptability, and creative expansion."),
        ("Truth", "Light", "Illumination, revelation, and integrity."),
        ("Mystery", "Shadow", "The unknown, the hidden, the fertile void.")

    ]


    return [

        {

```

```

        "id": i,
        "symbol": sym,
        "element": elem,
        "lore": lore,
        "rotation": i * 40
    }
    for i, (sym, elem, lore) in enumerate(symbols)
]

```

triangle_utils.py (inside root of Project)

```

import numpy as np

def isosceles_triangle(base=100, height=150, center=(200, 200), rotation=0):
    cx, cy = center
    points = np.array([
        [cx - base / 2, cy],
        [cx + base / 2, cy],
        [cx, cy - height]
    ])
    theta = np.radians(rotation)
    rot_matrix = np.array([
        [np.cos(theta), -np.sin(theta)],
        [np.sin(theta), np.cos(theta)]
    ])
    rotated = np.dot(points - [cx, cy], rot_matrix) + [cx, cy]
    return rotated.tolist()

```

```

def svg_triangle(points, symbol, element, name, lore):
    point_str = " ".join([f"{x},{y}" for x, y in points])
    return f"""
    <g class="triangle-group" onclick="streamlit.setComponentValue('{name}')">
        <polygon points="{point_str}" class="triangle"
            data-symbol="{symbol}"
            data-element="{element}"
            data-name="{name}"
            data-lore="{lore.replace('"' , '&quot;')}" />
    </g>
    """

def render_symbolic_card(meta):
    return f"""
    <div class="symbolic-card">
        <h3>{meta['symbol']} ({meta['element']})</h3>
        <p>{meta['lore']}</p>
    </div>
    """

```